# Contents


**1 Introduction (Alys z5418984) 3**
- 1.1 Problem . . . 3
- 1.2 Data Availability Sampling . . . 3
- 1.3 PeerDAS in Ethereum . . . 4
  - 1.3.1 Proto-Danksharding . . . 4
  - 1.3.2 PeerDAS . . . 4
- 1.4 Objectives . . . 4
- 1.5 Outline . . . 4

**2 Background Information (Alys z5418984) 5**
- 2.1 Notation . . . 5
- 2.2 Reed-Solomon Codes and Roots of Unity . . . 5
  - 2.2.1 Erasure Codes . . . 5
  - 2.2.2 Reed-Solomon Codes . . . 5
  - 2.2.3 Roots of Unity . . . 6
  - 2.2.4 Lagrange Basis Polynomial . . . 6
  - 2.2.5 Subgroups and Cosets . . . 6
  - 2.2.6 Reverse Bit Ordering . . . 6
- 2.3 Groups and d-BSDH Assumption . . . 7
  - 2.3.1 Bilinear Groups and Notation . . . 7
  - 2.3.2 d-BSDH Assumption . . . 7
- 2.4 KZG Commitments . . . 7
  - 2.4.1 Setup and Commitments . . . 7
  - 2.4.2 Opening and Verifying . . . 8
- 2.5 Erasure Code Commitments . . . 9

**3 PeerDAS Preliminaries (Alys z5418984) 9**
- 3.1 Erasure Code Commitment Scheme . . . 9
  - 3.1.1 Single Row Commitment Scheme . . . 9
  - 3.1.2 Full Commitment Scheme . . . 10
- 3.2 Security Analysis . . . 11

**4 PeerDAS Implementation 12**
- 4.1 Reed Solomon (Dominic z5163257) . . . 12
- 4.2 Polynomial Implementation (Dominic z5163257) . . . 12
- 4.3 KZG (Tim Faithfull z5479667) . . . 13
  - 4.3.1 Trusted Setup . . . 13
  - 4.3.2 Evaluation Domain . . . 13
  - 4.3.3 Polynomial Parameters . . . 13
  - 4.3.4 Committing . . . 13
  - 4.3.5 Opening . . . 14
  - 4.3.6 Verification . . . 14
  - 4.3.7 Erasure Recovery . . . 14
- 4.4 Networking Layer (Josh z5590080) . . . 14

4.4.1 Overview . . . 14
4.4.2 Disperser . . . 15
4.4.3 DA Node . . . 15
4.4.4 Verifier and Sampler . . . 15
4.4.5 Subnet Registry . . . 16
4.4.6 KZG Integration . . . 16
4.4.7 Parameter Selection . . . 16
4.4.8 Testing . . . 17
4.5 Benchmarking (Yang z5559793) . . . 18
4.5.1 Encoding (`encode_kzg`) . . . 18
4.5.2 Verification (`verify_kzg`) . . . 19
4.5.3 Pairing counts (`pairing_counts`) . . . 19
4.5.4 Running and configuring benchmarks . . . 19
4.5.5 Experimental parameters for the tables below . . . 19
4.5.6 Encode timing results . . . 20
4.5.7 Verify timing results . . . 20
4.5.8 Pairing-count model (tabular) . . . 21

# 1 Introduction (Alys z5418984)

## 1.1 Problem

As the use of blockchain and cryptocurrency continues to grow, so do the amount transactions that need to be processed per second in order to keep up with demand. This leads to a growing problem of scalability and data handling. As blockchains are distributed systems, it is imperative that each person within the blockchain with reasonable computational power is able to participate.

Within a blockchain, data is constructed as a sequence of blocks. Each block contains a header and content. In order to participate, clients either join as full nodes (store/verify whole block) or light nodes, which only store the headers. The issue is that light nodes are not able to verify that all of the data within a block is valid or present on the header alone. Light nodes rely on full nodes to detect and inform them of fraudulent occurrences, such as adversaries providing them with headers of malformed data. This inspection/communication is known as a fraud-proof.

A fraud-proof means a full node can convince a light node that a header and content do not comprise a valid block. The issue is that if an adversary withholds the block's content, the full node is not able to verify its validity on the header alone. They are able to alert the light node that the block is malformed, but not that the content is not available [1]. This is the data availability problem: light nodes need a way to verify that a block's full content is available without downloading it themselves.

## 1.2 Data Availability Sampling

Data Availability Sampling (DAS) schemes allows clients to verify the availability of data on a peer-to-peer network. They aim to solve the issue detailed above. This is done by encoding, committing, and storing data on different nodes on the network. Clients can then query random positions of the encoding from these nodes and check them against the committed version. If a sufficient amount of random samples all verify against the commitment, the clients can assume with high probability that the data is available. DAS must satisfy three key properties

1. *Completeness*: Verifiers possessing a valid commitment and examining a valid encoding should be able to determine that the code is fully available

2. *Soundness*: If enough clients accept, then some data can be reconstructed from their transcripts to ensure availability

3. *Consistency*: Ensuring clients with the same commitments will always reconstruct the same data

## 1.3 PeerDAS in Ethereum

Ethereum is one of the most popular global blockchains. This popularity means they are constantly dealing with the issue of scalability. Increasing the data handled by the network could possible overwhelm the nodes that are crucial for decentralisation. Therefore, they are in the process of implementing the DAS scheme PeerDAS.

### 1.3.1 Proto-Danksharding

In EIP 4844, Ethereum introduced blob-carrying transaction types to assist in DAS. Under EIP-4844, data is packaged into blobs rather than stored directly in transaction calldata. A blob is a new data structure designed to enhance blockchain scalability. Blobs are large packets of data that can be included in blocks that are stored by nodes for 18 days. A blob consists of 4096 field elements, representing 128KB of data. [2]

### 1.3.2 PeerDAS

In PeerDAS, data is first split into blobs, which are then individually extended using Reed-Solomon code. Each blob is assigned a polynomial of some degree $kD-1$ and are then evaluated at $Dn$ points to get an extended blob. These extended blobs are then arranged into a matrix; one row is an extended blob. The clients then download the KZG commitments for each extended blob. PeerDAS then uses sampling to check availability: each client downloads random columns of the matrix and verifies their KZG opening proofs against the commitments.

## 1.4 Objectives

In this project, we set out to implement PeerDAS using Python. We have split the implementation into Reed-Solomon, KZG and Networking. We also benchmarked ADD IN BENCHMARKING

## 1.5 Outline

This paper has been split into two main parts; theory and implementation. Section 2 covers the mathematical background including finite fields, Reed-Solomon codes, roots of unity, and KZG polynomial commitments. Section 3 describes the PeerDAS protocol, including the erasure code commitment scheme and the full matrix construction. Section 4 details our Python implementation of Reed-Solomon encoding, KZG proof generation and verification, and the sampling protocol. Section 5 presents benchmarking results across varying blob counts, batch sizes, and sample counts, with analysis of performance against theoretical complexity.

# 2 Background Information (Alys z5418984)

## 2.1 Notation

An element $z$ that has been randomly, uniformly sampled from a finite set $Z$ is written as $z \xleftarrow{\$} Z$. $y := A(x)$ means that $A$ is deterministic and is run on input $x$ and output $y$. However, if $A$ is probabilistic, we write $y := A(x; \rho)$ if $\rho$ is to be explicitly stated, $y \leftarrow A(x)$ if not. If $y$ is a possible output of $A$ using input $x$, $y \in A(x)$ is used. The set of the first $r$ natural numbers is denoted by $[r] = \{1, ..., r\} \subseteq \mathbb{N}$. Standard cryptography notation applies such as the security parameter $\lambda$, probabilistic polynomial-time (PPT) algorithms, and negligible functions.

## 2.2 Reed-Solomon Codes and Roots of Unity

### 2.2.1 Erasure Codes

An erasure code takes a message of $k$ symbols and expands it into a codeword of $n$ symbols, where $n > k$, to create redundancy. Formally, it is "an encoding function $\mathcal{C} : \Gamma^k \to \Lambda^n$ that maps messages over an alphabet $\Gamma$ to codewords over an alphabet $\Lambda$" [3]. An important property of erasure encoding is that you can reconstruct the message when provided with any $t < n$ symbols of a codeword. We treat the erasure code as a set $\mathcal{C} \subseteq \Delta^n$ given by $\mathcal{C} = \mathcal{C}(\Gamma^k)$.

### 2.2.2 Reed-Solomon Codes

Reed-Solomon (RS) is built upon erasure code and uses polynomial evaluation. It ensures that when data is transferred over a network, the data can still be recovered even if it has become corrupted. It is defined over a finite field $\mathbb{F}$. In PeerDAS, typically $\mathbb{F} = \mathbb{Z}_q$ for some large prime $q$, is parameterised by an integer $d \in \mathbb{N}$ and a set $\mathcal{L} \subseteq \mathbb{F}$. The Reed-Solomon code contains all codewords of the form $(f(x))_{x \in \mathcal{L}}$ where $f$ is a polynomial $f \in \mathbb{F}[X]$ of degree strictly less than $d$. We can interpret codewords as;

- Vectors in $\mathbb{F}^{|\mathcal{L}|}$ with implicit ordering on $\mathcal{L}$
- Maps $f : \mathcal{L} \to \mathbb{F}$

Reed-Solomon code is denoted by $\mathcal{RS}[d, \mathcal{L}, \mathbb{F}]$, meaning it is the set of all codewords. Reed-Solomon works by taking $d$ field elements, the raw data, and interpreting them as the coefficients of a polynomial:

$$f(X) = a_0 + a_1 X + a_2 X^s + ... + a_{d-1} X^{d-1}$$

Then using a set $\mathcal{L}$ of evaluation points where $\mathcal{L} > d$, we can compute $f(x)$ for each $x \in \mathcal{L}$. This list of evaluated points then become our codeword. Reed-Solomon is a Maximum Distance Separable (MDS) code, meaning that there is built-in redundancy that allows you to reconstruct the data if only $d$ points of the $n$ total evaluated points remain.

### 2.2.3 Roots of Unity

Roots of unity are complex numbers, $z$, that when raised to the power of some positive integer$n$, equal $\pm 1$, $z^n = 1$. In a finite field $z^m = 1$ can have up to $m$ solutions, which are called the $m$-th roots of unity. For a finite field $\mathbb{F}$, let $\omega$ be a primitive (can generate all other roots through repeated multiplication) root of unity such that $\omega^m = 1$ and $\omega^r \neq 1$ for all $1 \leq r < m$. The group generated by $\omega$ with cardinality $m$ is denoted as $\mathbb{H} = \{\omega^1, ..., \omega^m\}$ and is implicitly ordered.

### 2.2.4 Lagrange Basis Polynomial

The Vanishing Polynomial is a polynomial that evaluates to zero for all $\omega \in \mathbb{H}$. The formula for this is

$$z_{\mathbb{H}}(X) = X^m - 1$$

as all all elements of $\mathbb{H}$ satisfies $\omega^m = 1$, so $z_{\mathbb{H}}(\omega^i) = \omega^{im} - 1 = 0$ for all $\omega$. The Lagrange Polynomial Interpolation is a method of using other points on a function to find the value of any function at any given point without the function itself. The Lagrange basis polynomial is used in the Lagrange Polynomial Interpolation, as it is a polynomial that isolates a single point, meaning it is only equal to 1 at a single point $\omega^j$ of $\mathbb{H}$. The formal definition is

$$\lambda_i(X) = \frac{\omega^i}{m} \cdot \frac{X^m - 1}{X - \omega^i}$$

### 2.2.5 Subgroups and Cosets

A subgroup $\mathbb{H}_j$ is a subset of $\mathbb{H}$ that is formed by performing the same mathematical operation. Let $m = 2^t$ for some $t$. For each $j \in [t]$, these exists some $\mathbb{H}_j$ with size $2^{t-j}$ of $\mathbb{H}$ where $\mathbb{H}_j = \{\omega^i \in \mathbb{H} | i \equiv 0 \mod 2^j\}$. The subgroup $\mathbb{H}_j$ consists of elements whose index is divisible by $2^j$. A coset is a subset of a group $\mathbb{H}$ that is created by multiplying each element of the subgroup $\mathbb{H}_j$ by a fixed element $\omega^i \in \mathbb{H}$. For any $0 \leq s < 2^j$, the set $\mathbb{H}_{j,s} = \omega^s \mathbb{H}_j = \{\omega^i \in \mathbb{H} | i \equiv s \mod 2^j\}$ is the coset of $\mathbb{H}_j$. This multiplies $\mathbb{H}_j$ by some $\omega^s$ to partition $\mathbb{H}$ into equal sized, disjointed chunks. Both subgroups and cosets have sparse representations of their vanishing and Lagrange polynomials, making proofs computationally cheaper. For a set $r = 2^{t-j}$,

$$z_{j,s}(X) = X^r - \omega^{sr} \text{ and } \lambda_i^{s,r}(X) = \frac{\omega^i}{r\omega^{s(r-1)}} \cdot \frac{X^r - \omega^{sr}}{X - \omega^{si}} \text{ for all } i \in [r]$$

This means that a node only has to compute the vanishing polynomial and the Lagrange interpolation for a coset instead of the whole of $\mathbb{H}$.

### 2.2.6 Reverse Bit Ordering

If the elements of $\mathbb{H}$ are stored in there natural order $\{\omega, \omega^2, ..., \omega^m\}$, the cosets become scattered in the array. This would make working with them difficult, as calling different points of the set is messy. Reverse bit ordering uses the binary

representation of a set of roots and reverses them. Splitting the elements in this ordering into chunks of size $2^{t-j}$ where $j \in [t]$ will result in the chunks corresponding to cosets $\mathbb{H}_{j,s}$ The notation for an element in this ordering is $\tilde{w}_i$.

## 2.3 Groups and d-BSDH Assumption

### 2.3.1 Bilinear Groups and Notation

Elements of a bilinear group are denoted implicitly as $[a]_\gamma = a\mathcal{P}_\gamma$. In essence, $a_\gamma$ means the group element you get by multiplying the generator $\mathcal{P}_\gamma$ by some scalar $a$. A bilinear group allows a specific mapping between groups, enabling mathematical operations across the different groups. Formally, a "bilinear group $\mathcal{G}$ is a tuple $\mathcal{G} = (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, \mathcal{P}_1, \mathcal{P}_2)$ where $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ are groups of prime order $q$ with generators $(\mathcal{P}_1, \mathcal{P}_2)$, respectfully, and $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is an efficient computable, non-degenerate bilinear map" [3]. This means that the bilinear map $e$ is able to take one element of $\mathbb{G}_1$ and one from $\mathbb{G}_2$ and produce some element of $\mathbb{G}_T$. A

### 2.3.2 d-BSDH Assumption

It is important to note that bilinear groups are an example of type-3 pairing groups. This means there is no efficient isomorphism that exists between groups. This is important as it ensures that security proofs remain valid. This is important as it allows us to use stronger security assumptions, such as the d-BSDH Assumption. KZG commitments and PeerDAS security proofs both rely on the d-BSDH Assumption, which assumes that given all powers $\tau$ in both $\mathbb{G}_{\not\vdash}$ and $\mathbb{G}_2$, it is computationally infeasible to compute

$$[\frac{1}{\tau - c}]_T$$

## 2.4 KZG Commitments

KZG Commitments is a polynomial commitment scheme that, for a polynomial of degree $d \in \mathbb{N}$, allows for

- Committing a polynomial $f \in \mathbb{Z}_q[X]$ without revealing it to produce a single group element
- Generate opening proofs that can convince a verifier that for some evaluation points $x \in \mathbb{Z}_q$, $f(x) = y$ without revealing the polynomial

### 2.4.1 Setup and Commitments

The setup algorithm is a trusted setup that first samples a random secret $\tau$ and outputs its power the groups. The commitment algorithm uses the curve points generated in the setup to then evaluate $f$ over $\mathbb{G}_1$.

- $\mathsf{KZG.Setup}(1^\lambda, 1^d) \to \mathsf{ck}$:
  1. Generate $\mathcal{G} \leftarrow \mathsf{PGGen}(1^\lambda)$, where $\mathcal{G} = (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, \mathcal{P}_1, \mathcal{P}_2)$.
  2. Sample $\tau \xleftarrow{\$} \mathbb{Z}_q$ and set commitment key $\mathsf{ck} := \left(\mathcal{G}, d, ([\tau^i]_1)_{i=0}^d, ([\tau^i]_2)_{i=0}^d\right)$.
- $\mathsf{KZG.Com}(\mathsf{ck}, f) \to \mathsf{com}$ for $f \in \mathbb{Z}_q[X]$ with $\deg f \leq d$:
  1. Write $f(X) = \sum_{i=0}^d f_i X^i$.
  2. Set $\mathsf{com} = [f(\tau)]_1 = \sum_{i=0}^d f_i [\tau^i]_1$.

Figure 1: KZG Setup and Com algorithms taken from WZ2024 [3]

### 2.4.2 Opening and Verifying

Once the setup and commitment is done, it is time to create opening proofs for the verifier, which prove that a value in the commitment exists. In PeerDAS, polynomials are not opened individually but instead over cosets $\mathbb{H}_{j,s}$ of a subgroup, size $r = 2^{t-j}$ comprised of roots of unity $\mathbb{H}$, size denoted as $m = 2^t$.

- $\mathsf{KZG.MultiOpen}(\mathsf{ck}, f, m, r, \hat{s}) \to \pi$ for $f \in \mathbb{Z}_q[X]$ with $\deg f \leq d$, $m = 2^t$, $r = 2^{t-j}$, and $0 \leq \hat{s} < 2^j$:
  1. Set $\hat{m}_i := f(\tilde{\omega}_{\hat{s}r+i})$ for each $i \in [r]$.
  2. Let $s$ be such that $0 \leq s < 2^j$ and $\omega^s \mathbb{H}_j = \{\tilde{\omega}_{\hat{s}r}, \ldots, \tilde{\omega}_{\hat{s}r+(r-1)}\}$.
  3. Set $I(X) := \sum_{i=1}^r \hat{m}_i \lambda_i^{j,s}(X) \in \mathbb{Z}_q[X]$.
  4. Let $Q(X) := (f(X) - I(X))/z_{j,s}(X)$ and observe that $Q \in \mathbb{Z}_q[X]$ and $\deg Q \leq d$.
  5. Write $Q(X) = \sum_{i=0}^d Q_i X^i$ and set $\pi := [Q(\tau)]_1 = \sum_{i=0}^d Q_i [\tau^i]_1$.
- $\mathsf{KZG.MultiVer}(\mathsf{ck}, \mathsf{com}, m, r, \hat{s}, \hat{m}, \pi) \to b$ for $\hat{m} \in \mathbb{Z}_q^l$ and $m = 2^t$, $r = 2^{t-j}$, and $0 \leq \hat{s} < 2^j$:
  1. Let $s$ be such that $0 \leq s < 2^j$ and $\omega^s \mathbb{H}_j = \{\tilde{\omega}_{\hat{s}r}, \ldots, \tilde{\omega}_{\hat{s}r+(r-1)}\}$.
  2. Set $I(X) := \sum_{i=1}^r \hat{m}_i \lambda_i^{j,s}(X) \in \mathbb{Z}_q[X]$.
  3. If $e\left(\mathsf{com} - [I(\tau)]_1, [1]_2\right) = e\left(\pi, [z_{j,s}(\tau)]_2\right)$, return $b := 1$. Otherwise, return $b := 0$.

Figure 2: KZG MultiOpen and MultiVer algorithms taken from WZ2024 [3]

The quotient $Q(x)$ is the main part of the opening algorithm. It works as $f(X) - I(X))$ will be zero at every point in the coset. The denominator, $z_{j,s}(X)$ is the vanishing polynomial that will be zero at only the coset points. Dividing the numerator and denominator will result in a polynomial with no remainder if $f(X) = I(X)$. If there is an was an issue in the prover, then $I(X)$ would not equal $f(X)$ and the vanishing polynomial would not divide $f(X) - I(X)$ cleanly. The quotient polynomial can then be committed using $\tau$, creating a proof for the coset. The verifier then independently builds $I(X)$, using $\hat{m}$ and evaluates it using $\tau$. The pairing equation in the verification is only valid if $\hat{m}$ is valid.

## 2.5 Erasure Code Commitments

As defined above, erasure coding involves breaking a large block of data into smaller fragments, expanding and encoding them with redundant data and storing them. Erasure Code Commitments a form of commitment scheme that involves committing to codewords of an erasure code. In PeerDAS, this is used to commit to an extended blob. The definition of a Code Commitment Scheme is:

**Definition 2** (Erasure Code Commitment Scheme). Let $\mathcal{C} : \Gamma^k \to \Lambda^n$ be an erasure code. An erasure code commitment scheme for $\mathcal{C}$ is a tuple $\mathsf{CC} = (\mathsf{CC.Setup}, \mathsf{CC.Com}, \mathsf{CC.Open}, \mathsf{CC.Ver})$ of PPT algorithms, with the following syntax:

- $\mathsf{CC.Setup}(1^\lambda) \to \mathsf{ck}$ : takes as input the security parameter and outputs a commitment key ck.
- $\mathsf{CC.Com}(\mathsf{ck}, m) \to (\mathsf{com}, St)$ : takes as input a commitment key ck and a string $m \in \Gamma^k$, and outputs a commitment com and a state $St$.
- $\mathsf{CC.Open}(\mathsf{ck}, St, j) \to \pi$ : takes as input a commitment key ck, a state $St$, and an index $j \in [n]$, and outputs an opening $\pi$.
- $\mathsf{CC.Ver}(\mathsf{ck}, \mathsf{com}, j, \hat{m}_j, \pi) \to b$ : is deterministic, takes as input a commitment key ck, a commitment com, and index $j \in [n]$, a symbol $\hat{m}_j \in \Lambda$, and an opening $\pi$, and outputs a bit $b \in \{0, 1\}$.

Figure 3: Code Commitment Scheme from WZ2024 [3], based on HSW2023 [1]

# 3 PeerDAS Preliminaries (Alys z5418984)

PeerDAS uses a code commitment scheme in it. As with a DAS scheme, if the code commitment scheme satisfies security, then PeerDAS does as well. We will expand on the security considerations later in this section. As defined above, PeerDAS uses blobs, which are made by splitting the given data into equal-sized chunks. Each blob is extended and KZG commitment (now denoted as extended blobs) and then used to create a matrix where they represent one row within this matrix (the codeword). That is the general overview, but it will be expanded on below.

## 3.1 Erasure Code Commitment Scheme

### 3.1.1 Single Row Commitment Scheme

The reason we start by looking at only a single row at first is that the matrix is comprised of extended blobs, so instead of jumping into the whole matrix, its easier to start by looking at one row.

We use Reed-Solomon for the erasure code, denoted by $\mathcal{RS}_{\text{row}}$. Each blob (message) is split into $k$ cells, which represent a chunk of $D$ field elements. Therefore, each blob contains a total of $kD$ field elements that are interpreted as evaluations of a polynomial $f$ of degree less than $kD$. The extended blob (codeword) is comprised of $n$ cells where $k, n \in \mathbb{N}$ and $n > k$ and is produced by evaluating

$f$ at all $nD$ roots of unity. The extra $n-k$ cells are the redundancy, allowing us to reconstruct the message as long as we have $k$ cells via Lagrange interpolation. Clients download entire cells rather than individual field elements. $\mathcal{RS}_{\text{row}}$ is therefore defined by grouping $D$ consecutive elements together into a single symbol or cell, giving codewords in $(Z_q^D)^n$.

Now that we understand the erasure code for a single row, we can look at how that code is committed. We denote this extended blob erasure code commitment scheme as $CC_{\text{row}}$. The commitment scheme used in this case is the KZG polynomial scheme discussed above. The correctness of the $CC_{\text{row}}$ scheme comes from KZG.

- $\mathsf{CC}_{\mathsf{row}}.\mathsf{Setup}(1^\lambda) \to \mathsf{ck}$
  1. Sample $\mathsf{ck} \leftarrow \mathsf{KZG.Setup}(1^\lambda, 1^d)$ for $d = kD - 1$.
- $\mathsf{CC}_{\mathsf{row}}.\mathsf{Com}(\mathsf{ck}, m) \to (\mathsf{com}, St)$ for $m \in \mathbb{Z}_q^{Dk}$
  1. Let $f \in \mathbb{Z}_q[X]$ with $\deg f < Dk$ such that $f(\tilde{\omega}_i) = m_i$ for all $i \in [Dk]$.
  2. Set $\mathsf{com} := \mathsf{KZG.Com}(\mathsf{ck}, f)$ and $St := f$.
- $\mathsf{CC}_{\mathsf{row}}.\mathsf{Open}(\mathsf{ck}, St, j) \to \pi$ for $j \in [n]$
  1. Compute $\pi := \mathsf{KZG.MultiOpen}(\mathsf{ck}, f, Dk, D, j-1)$.
- $\mathsf{CC}_{\mathsf{row}}.\mathsf{Ver}(\mathsf{ck}, \mathsf{com}, j, \hat{m}, \pi) \to b$ for $i \in [n]$ and $\hat{m} \in \mathbb{Z}_q^D$
  1. Set $b := \mathsf{KZG.MultiVer}(\mathsf{ck}, \mathsf{com}, Dk, D, j-1, \hat{m}, \pi)$.

Figure 4: Single Row Commitment Scheme from WZ2024 [3]

### 3.1.2 Full Commitment Scheme

Now that we understand erasure code commitment for a single row, or extended blob, it just becomes a matter of applying that same logic to all extended blobs in the matrix. We are taking a collection of 1D codes and turning it into a 2D matrix structure. Let $l \in \mathbb{N}$ be the number of blobs the data is split into (or the number of rows). This means the total data is now $\mathbb{Z}_q^{Dkl}$. The encoding works by

1. Split data into $l$ blobs $m_1, ..., m_l$ where $m_i \in \mathbb{Z}_q^{Dk}$
2. Each blob is encoded independently using the $\mathcal{RS}_{\text{row}}$ defined in the previous section. This creates a matrix of $ln$ cells. We now have $c_1, ...c_l$ extended blobs where $x_i \in (\mathbb{Z}_q^D)^n$
3. We then have to redefine symbols as columns, as this is what each node downloads. Let $j$ be the column index for which we do this redefinition, $c_{1,j}, ..., c_{l,j} \in (\mathbb{Z}_q^D)^l$, which is used to denote the $j$the cell of $c_i$

This result in the Reed Solomon Code for the whole matrix $\mathcal{RS}_{\text{full}} \subset ((\mathbb{Z}_q^D)^l)^n$

The full commitment scheme works by committing each row independently but opening them column by column. In the verification, a column is only valid if every row's cell at that column position verifies correctly. Correctness follows from $CC_{\text{row}}$.

- $\mathsf{CC}_{\mathsf{full}}.\mathsf{Setup} = \mathsf{CC}_{\mathsf{row}}.\mathsf{Setup}$
- $\mathsf{CC}_{\mathsf{full}}.\mathsf{Com}(\mathsf{ck}, m) \to (\mathsf{com}, St)$ for $m \in \mathbb{Z}_q^{Dk\ell}$
  1. Parse $m = m_1 \parallel \cdots \parallel m_\ell$ with $m_i \in \mathbb{Z}_q^{Dk}$ for all $i \in [\ell]$.
  2. For each $i \in [\ell]$, compute $(\mathsf{com}_i, St_i) := \mathsf{CC}_{\mathsf{row}}.\mathsf{Com}(\mathsf{ck}, m_i)$.
  3. Set $\mathsf{com} = (\mathsf{com}_1, \ldots, \mathsf{com}_\ell)$ and $St := (St_1, \ldots, St_\ell)$.
- $\mathsf{CC}_{\mathsf{full}}.\mathsf{Open}(\mathsf{ck}, St, j) \to \pi$ for $j \in [n]$
  1. Parse $St = (St_1, \ldots, St_\ell)$.
  2. For each $i \in [\ell]$, compute $\pi_i := \mathsf{CC}_{\mathsf{row}}.\mathsf{Open}(\mathsf{ck}, St_i, j)$.
  3. Set $\pi := (\pi_1, \ldots, \pi_\ell)$.
- $\mathsf{CC}_{\mathsf{full}}.\mathsf{Ver}(\mathsf{ck}, \mathsf{com}, j, \hat{m}, \pi) \to b$ for $j \in n$ and $\hat{m} \in \left(\mathbb{Z}_q^D\right)^\ell$
  1. Parse $\mathsf{com} = (\mathsf{com}_1, \ldots, \mathsf{com}_\ell)$, $\pi = (\pi_1, \ldots, \pi_\ell)$, and $\hat{m} = (\hat{m}_1, \ldots, \hat{m}_\ell)$.
  2. If $\mathsf{CC}_{\mathsf{row}}.\mathsf{Ver}(\mathsf{ck}, \mathsf{com}_i, j, \hat{m}_i, \pi_i) = 1$ for all $i \in [\ell]$, set $b := 1$. Otherwise, set $b := 0$.

Figure 5: Single Row Commitment Scheme from WZ2024 [3]

## 3.2 Security Analysis

The security of PeerDAS rests on two properties within the erasure code commitment scheme: position-binding and code-binding. Both of these properties were formally proven by Wagner and Zapico [3], which illustrates that a malicious block proposer cannot convince a honest light node that data is available when it is not. The two properties are:

- Position-binding: A commitment cannot be opened to two different values at the same position. This means a prover can not produce two valid cell proofs for the same cell index that have different cell content
- Code-binding: Any set of valid openings produced by an adversary must be consistent with some valid codeword of the Reed-Solomon Code. This means an adversary cannot construct valid cell proofs that, when combined, do not create a valid blob.

Both of these properties are proven to hold under the d-BSDH assumption in the algebraic group model, and PeerDAS is deemed secure.

# 4 PeerDAS Implementation

## 4.1 Reed Solomon (Dominic z5163257)

To implement Reed Solomon, we created a `Block` class which holds the raw data and serves as the input to the Reed-Solomon encoder. Blocks have a fixed length `block_len`, with raw data smaller than the block length being padded with zeros. Due to this padding, we also store an `underlying_len` variable per class instance, so that after decoding we do not confuse the padded zeros with the original data.

The Reed-Solomon encoding is implemented as a class. On initialisation, the class takes in parameters $k$ and $n$ representing the data block length and the intended expansion respectively. We use the Galois finite field of 256 elements as each byte of the message will naturally map into this field. To encode the data we define the evaluation points of a polynomial with the message bytes as coefficients to be powers of a primitive element of the field, up to $n$. Encoding involves taking the bytes of the block and evaluating this polynomial at each of the evaluation points, producing an encoded block of size $n$.

## 4.2 Polynomial Implementation (Dominic z5163257)

To support KZG polynomial commitment, we implemented a polynomial class which handles arbitrary arithmetic compatible coefficient types. The implementation stores the coefficients as a list, in ascending order, i.e. $[a_0, a_1, \ldots, a_n]$ corresponds to the polynomial

$$\sum_{k=0}^{n} a_k X^k.$$

The standard vector operations such as addition, multiplication, and scalar multiplication are supported. Additionally, we support polynomial division with remainder and Lagrange interpolation.

To implement interpolation, we do the standard construction where if you have points

$$(x_0, y_0), \ldots, (x_n, y_n)$$

and want to obtain the polynomial $f$, where $f(x_i) = y_i$ we define the basis polynomials

$$L_i(x) = \prod_{\substack{0 \leqslant j \leqslant n \\ j \neq i}} \frac{x - x_j}{x_i - x_j}$$

then define

$$f(x) := \sum_{i=0}^{n} L_i(x) y_i.$$

This construction works because for $x_i$, $L_i(x_i) = 1$ and for $i \neq j$, $L_i(x_j) = 0$, hence $f(x_i) = y_i$ for all $0 \leqslant i \leqslant n$ as intended.

## 4.3 KZG (Tim Faithfull z5479667)

The KZG implementation is split across two files: `KZG.py`, which contains the core cryptographic primitives, and `kzg_context.py`, which acts as an adapter bridging the KZG logic to the rest of the PeerDAS network. The latter is covered in the networking portion of this report.

### 4.3.1 Trusted Setup

The trusted setup is handled by `KZGSetup.generate()`, which picks a random secret scalar `tau`, computes its powers on both the G1 and G2 elliptic curves up to a configurable `max_degree`, and discards `tau`. The resulting Structured Reference String (SRS) stores these curve points $[\tau^0]_1, [\tau^1]_1, \ldots$ on G1 and equivalently on G2, along with a precomputed $[\tau^D]_2$ that is used in every verification call.

### 4.3.2 Evaluation Domain

`RootsOfUnity` manages the set of points at which polynomials are evaluated. It is constructed with a primitive root of unity `omega` and a domain size $m$, which must be a power of two. Elements are stored in reverse-bit order (RBO) so that any contiguous block of $D$ domain points forms one cell's coset. The class implements FFT and inverse FFT over the scalar field using a Cooley-Tukey butterfly, along with `coset_fft` / `coset_ifft` variants that evaluate over a shifted domain using $\beta = 7$, these are used during erasure recovery.

### 4.3.3 Polynomial Parameters

`KZG.py` defines three core constants: `D_CELL_SIZE` = 64 (evaluations per cell), `K_CELLS_BLOB` = 64 (cells per blob), and `N_CELLS_EXT` = 128 (cells per extended blob, equal to 2× `K_CELLS_BLOB`). The polynomial committed to for each blob has degree less than $D \times k$, and is evaluated at all $n$ extended domain points to produce the full set of cells. The value $D \times k = 4096$ was chosen as it is a power of two, satisfying the FFT requirement, while landing at roughly 128KB per blob the PeerDAS data throughput target. The $64 \times 64$ factoring splits this into cells small enough for validators to download individually, and `N_CELLS_EXT` = 128 provides exactly 2× Reed-Solomon redundancy, meaning any half of the extended blob suffices for full reconstruction.

### 4.3.4 Committing

`CCrow.commit()` takes $D \times k$ raw field elements for a single blob, Lagrange-interpolates them to find the unique polynomial $f$ passing through those values at the first $k$ coset-aligned domain points, and commits to it by computing

$[f(\tau)]_1$ using the SRS. `CCfull.commit()` repeats this for every blob row and then immediately re-evaluates the polynomial at every column so the cells stored in the matrix are always consistent with what the verifier expects.

#### 4.3.5 Opening

`KZGProver.multi_open()` produces a proof for a single cell at `cell_index`. It evaluates $f$ at the $D$ coset points belonging to that cell, Lagrange-interpolates those values into a polynomial $I(X)$, and computes the quotient

$$Q(X) = \frac{f(X) - I(X)}{z(X)}$$

where $z(X) = X^D - \texttt{coset\_root}$ is the cell's vanishing polynomial. The proof is the KZG commitment $[Q(\tau)]_1$, returned alongside the cell values. `CCfull.open()` repeats this for every blob row at the same column index.

#### 4.3.6 Verification

`KZGVerifier.multi_verify()` checks one cell opening using two pairings against the commitment, the reconstructed $[I(\tau)]_1$, the proof $[Q(\tau)]_1$, and the precomputed $[\tau^D]_2$. `CCfull.verify_column()` runs this check once per blob row.

A `batch_verify()` method is also implemented that collapses any number of (row, column) proof checks into just two pairings by deriving a random scalar $r$ from a SHA-256 hash of all inputs and taking a linear combination, this is available for use where throughput matters more than per-cell latency.

#### 4.3.7 Erasure Recovery

`reconstruct_extended_blob()` recovers all $n \times D$ evaluations when some cells are missing. It requires at least $k$ cells, half the extended blob to reconstruct. Given the available cells, it first builds the vanishing polynomial $Z(X)$ of all missing positions by multiplying in one linear factor per missing point. It then constructs $E(X)$ by interpolating the known evaluations with zeros substituted at missing positions, and forms the product $E(X) \cdot Z(X)$. Since $Z$ is zero on the entire domain $H$, direct division is not possible there; instead, both $E \cdot Z$ and $Z$ are evaluated over the shifted coset $\beta \cdot H$ using `coset_fft`, divided pointwise, and transformed back via `coset_ifft` to recover the polynomial $P(X) = E(X)$. A final FFT over $H$ yields all $n \times D$ recovered field elements.

### 4.4 Networking Layer (Josh z5590080)

#### 4.4.1 Overview

The networking layer implements the three core roles in PeerDAS: the *Disperser*, the *DA Node*, and the *Verifier*. All three are built on a shared `BaseNode` class that handles TCP communication using Python's `asyncio` library, allowing a single process to handle many concurrent connections without threads.

#### 4.4.2 Disperser

The Disperser is run by the block proposer. Given raw block data, it:

1. Splits the data into $\ell$ blobs (rows) and Reed-Solomon encodes each blob at rate $\frac{1}{2}$, producing $n$ cells per row.
2. Computes one KZG commitment per blob using the polynomial committed to that blob's systematic cells.
3. Computes one KZG multiproof per column using `CCfull.open()`, which opens the same column index across all $\ell$ blob polynomials simultaneously.
4. Sends each column to every DA node that custodies it, using the `SubnetRegistry` to find the relevant nodes.

All column sends are issued in a single `asyncio.gather()` call so network latency does not compound across columns.

#### 4.4.3 DA Node

Each DA node custodies a fixed subset of column indices, assigned at creation. When a `COLUMN` message is received, the node checks the column index against its custody set and discards any column outside it. Valid columns are stored in a two-level dictionary indexed by `block_id` and `col_index`, so a node can hold data for multiple blocks at once.

When a `SAMPLE_REQ` is received, the node either returns the stored column data and KZG proof as a `SAMPLE_RESP`, or returns `MSG_UNAVAILABLE` if it has no record of that block and column.

#### 4.4.4 Verifier and Sampler

The `Verifier` class only handles the TCP mechanics of sending sample requests and receiving responses. All sampling logic is in `sampler.py`.

The `Sampler` class implements the PeerDAS sampling protocol:

1. Randomly selects `sample_count` column indices from $[n]$.
2. For each selected column, picks a DA node from that column's subnet at random.
3. Issues all column requests concurrently using `asyncio.gather()`.
4. Verifies each response's KZG multiproof against the blob commitments by calling `verify_column()` on a `KZGContext` instance passed in at construction time.
5. Declares the block available if the fraction of verified columns meets a configurable `threshold` (default 1.0).

The `KZGContext` is injected into the `Sampler` as a parameter rather than being looked up internally. This keeps the sampler testable in isolation — unit tests pass a mock context whose `verify_column()` returns a controlled value, so sampling logic can be verified independently of the KZG computation.

#### 4.4.5 Subnet Registry

The `SubnetRegistry` maps column indices to the DA nodes that custody them. In production Ethereum PeerDAS, nodes self-select columns and advertise their choices via the discovery layer. In our implementation, custody sets are assigned statically at node creation. Each column can be held by multiple nodes, providing redundancy if a custodian goes offline.

#### 4.4.6 KZG Integration

The networking and sampler layers interact with the KZG commitment scheme through `KZGContext`, an adapter class in `src/commitments/kzg_context.py`. This keeps `KZG.py` entirely unmodified and isolates all serialisation concerns. $\mathbb{G}_1$ elliptic curve points are not JSON-serialisable, so `KZGContext` converts them to and from `"x_hex:y_hex"` strings for transmission over the wire.

A shared trusted setup is maintained via a process-level cache (`_get_kzg_ctx`). The disperser and verifier must use the same SRS — if constructed with different secrets $\tau$, the pairing check

$$e(C - [I(\tau)]_1, [1]_2) = e(\pi, [z(\tau)]_2)$$

would always fail. The cache ensures both roles share one `KZGContext` instance per $(n_{\text{blobs}}, n_{\text{cols}})$ parameter set.

#### 4.4.7 Parameter Selection

The key parameters are summarised in Table 1.

| Parameter | Demo value | Ethereum mainnet |
|---|---|---|
| $n_{\text{cols}}$ (total columns) | 8 | 128 |
| $n_{\text{blobs}}$ (rows per block) | 4 | up to 64 |
| $k$ (systematic cells, rate $\frac{1}{2}$) | $n_{\text{cols}}/2 = 4$ | 64 |
| $D$ (field elements per cell) | 1 | 64 |
| Custody columns per node | 2 | $\geq 2$ |
| Sample count | 4 of 8 | $\sim$75 of 128 |

Table 1: Network layer parameters

$n_{\text{cols}}$ is kept small for the prototype because the KZG trusted setup cost scales with domain size $m = n_{\text{cols}} \times D$. At $n_{\text{cols}} = 8$, $D = 1$, setup takes approximately 2 minutes in pure Python. The rate-$\frac{1}{2}$ coding rate ($k = n_{\text{cols}}/2$)

matches the PeerDAS specification and means any $k$ columns are sufficient to reconstruct the original data.

The sample count of 4 out of 8 columns is chosen so the demo exercises the sampling protocol without requesting all columns. In Ethereum's production parameters, a sample count of ∼75 of 128 is used so the probability of a light client accepting an unavailable block is at most $2^{-\lambda}$.

#### 4.4.8 Testing

Testing was implemented by Josh Allwood and covers unit tests, integration tests, and end-to-end tests.

**Unit Tests — Networking Layer** Unit tests are split into fast no-TCP tests and TCP integration tests. The no-TCP tests call internal methods directly to isolate logic from the network layer:

- `TestSubnetRegistry` — verifies that `nodes_for_column()` returns the correct custodians, handles shared custody, and returns an empty list for unknown columns.
- `TestDANodeHandleColumn` — verifies that columns within a node's custody set are stored correctly, and that columns outside the custody set are silently discarded.
- `TestDANodeHandleSampleReq` — verifies that `MSG_SAMPLE_RESP` is returned for present columns and `MSG_UNAVAILABLE` for absent ones.
- `TestEncodeMatrix` — verifies that `_encode_matrix` produces a matrix of shape $n_{\text{blobs}} \times n_{\text{cols}}$ with non-empty cells, valid hex commitments, and JSON-encoded proof lists.

**Unit Tests — Sampler** All sampler tests mock `Verifier.fetch_column()` so no TCP connections are opened. A mock `KZGContext` is passed to the `Sampler` so tests can control whether verification passes or fails independently of the network response. Tests cover: all columns verifying (`available=True`), KZG verification returning false (`available=False` with `failed_count` populated), no nodes responding, wrong message type handling, threshold behaviour at both boundaries, `sample_specific()` querying exactly the requested columns, and concurrent request firing.

**Integration Tests** End-to-end tests run the full pipeline over real TCP connections with real KZG commitments and proofs. A shared event loop is used across all tests rather than `IsolatedAsyncioTestCase`, which triggers a known Python 3.10 bug where asyncio's `repr` machinery recurses infinitely through circular future references during event loop teardown, causing a C-stack overflow and segfault. This bug is fixed in Python 3.11.

Tests are run at multiple parameter sizes to verify correctness across configurations:

| **Test** | $n_{\text{cols}}$ | $k$ | **What is verified** |
|---|---|---|---|
| Disperse and verify one column | 4 | 2 | Happy path, col 0 |
| All columns verify | 4 | 2 | All $n_{\text{cols}}$ proofs pass |
| Tampered cell fails | 4 | 2 | Security: bit-flip detected |
| Two blocks independent | 8 | 4 | Different commitments |
| Larger column size | 16 | 8 | Correctness at larger parameters |
| Missing block unavailable | 4 | 2 | `MSG_UNAVAILABLE` returned |

Table 2: Integration test coverage

The two-blocks test uses $n_{\text{cols}} = 8$ rather than 4 because at $n_{\text{cols}} = 4$, $k = 2$, meaning KZG commits to only 2 bytes per blob. Inputs sharing the same first 2 bytes produce identical commitments — not a bug, but a consequence of small parameters discovered through testing.

**Approximate Test Runtimes** KZG operations are expensive in pure Python as `py_ecc` uses Python-level elliptic curve arithmetic. The KZG context is cached per $(n_{\text{blobs}}, n_{\text{cols}})$ pair within a session so the trusted setup is only run once regardless of how many tests share the same parameters.

| $n_{\text{cols}}$ | Trusted setup | Integration test suite |
|---|---|---|
| 4 | ∼20s | ∼2 min |
| 8 | ∼2 min | ∼5 min |
| 16 | ∼10 min | ∼15 min |

Table 3: Approximate KZG runtimes in pure Python

## 4.5 Benchmarking (Yang z5559793)

We added a small benchmarking harness so that cryptographic costs of the PeerDAS prototype can be measured independently of TCP latency. Drivers live under `src/benchmarking/`; each driver logs timings for later analysis. Inputs use deterministic pseudo-random block bytes so experiments are repeatable, and wall-clock times are taken in milliseconds.

### 4.5.1 Encoding (`encode_kzg`)

This driver times `_encode_matrix` from the networking layer: Reed–Solomon extension of the blob matrix, one KZG commitment per blob, and one column multiproof per column index—the same work the disperser performs before columns are sent to DA nodes. For each pair $(\ell, n_{\text{cols}})$ of blob count and column count, the harness performs one untimed warmup call so that the cached trusted setup inside `_get_kzg_ctx` is not included in the reported average. Each

configuration records blob and column counts, block size, whether the real RS encoder is available, the repetition count, and the mean encode time per call.

#### 4.5.2 Verification (`verify_kzg`)

For each matrix geometry $(\ell, n_{\text{cols}})$, the driver encodes a fixed block once, then measures how long it takes to verify randomly sampled columns using the current naive path: one `verify_column` call per column, each performing $2\ell$ pairing checks (no batched verification). The default entrypoint sweeps several geometries and several values of $L$ (columns per round), averaging over multiple rounds for each tuple $(\ell, n_{\text{cols}}, L)$, so we obtain both mean time per round and mean time per single column verification. These figures quantify how sample count and blob count affect a light client that checks openings one column at a time.

#### 4.5.3 Pairing counts (`pairing_counts`)

This script does not time elliptic-curve operations; it writes symbolic counts only. For given $\ell$ and $L$, it records the number of pairings required by naive per-column verification ($2L\ell$) versus an idealised batched verifier that uses two pairings regardless of $L$ and $\ell$. Together with the per-column verification times from the `verify_kzg` driver, one can estimate an effective milliseconds-per-pairing figure and reason about how much headroom batching would provide.

#### 4.5.4 Running and configuring benchmarks

Each module is runnable as, for example, `python -m src.benchmarking.benchmarks.peerdas.encode_kzg`. Parameters such as matrix geometries, repetition counts, lists of $L$ values, and block sizes are edited in each module's `__main__` entry point (or by calling the exported `benchmark_*` functions from another script). Re-running a driver overwrites that driver's output file for a fresh run.

#### 4.5.5 Experimental parameters for the tables below

The tables report outputs from the three benchmark drivers (`encode_kzg`, `verify_kzg`, and `pairing_counts`). Encode and verify runs both use a 2048-byte block, the real Reed–Solomon encoder (`rs_available` = 1), and BLS12-381 pairings via `py_ecc` in pure Python on a Windows 10 development machine. Encode timings are mean milliseconds for `_encode_matrix` over three timed repetitions after one warmup per $(\ell, n_{\text{cols}})$ pair. Verification uses the default multi-geometry sweep from `verify_kzg`: matrices $(\ell, n_{\text{cols}}) \in \{(2,4), (2,8), (4,8), (4,16)\}$, $L \in \{1, 2, 4\}$ columns verified per sampling round (capped at $n_{\text{cols}}$), eight averaging rounds per configuration, naive per-column path. Tabulated times are rounded to two decimal places.

#### 4.5.6 Encode timing results

| $\ell$ | $n_{\text{cols}}$ | Mean encode (ms) |
|---|---|---|
| 2 | 4 | 708.28 |
| 2 | 8 | 3249.20 |
| 4 | 8 | 6473.44 |
| 4 | 16 | 29121.41 |

Table 4: Mean `_encode_matrix` time after warmup (`encode_kzg`, three repetitions, 2048 B block).

Table 4 shows that cost is dominated by KZG work: doubling $\ell$ at fixed $n_{\text{cols}}$ from $(2, 8)$ to $(4, 8)$ roughly doubles the mean encode time, while doubling $n_{\text{cols}}$ at $\ell = 4$ from $(4, 8)$ to $(4, 16)$ increases it by a factor of about 4.5, reflecting more column multiproofs and a larger SRS domain.

#### 4.5.7 Verify timing results

| $\ell$ | $n_{\text{cols}}$ | $L$ | Mean round (ms) | Mean per `verify_column` (ms) |
|---|---|---|---|---|
| 2 | 4 | 1 | 16838.06 | 16838.06 |
| 2 | 4 | 2 | 33153.72 | 16576.86 |
| 2 | 4 | 4 | 66334.99 | 16583.75 |
| 2 | 8 | 1 | 16637.14 | 16637.14 |
| 2 | 8 | 2 | 33344.88 | 16672.44 |
| 2 | 8 | 4 | 66439.80 | 16609.95 |
| 4 | 8 | 1 | 33170.36 | 33170.36 |
| 4 | 8 | 2 | 66612.60 | 33306.30 |
| 4 | 8 | 4 | 133109.80 | 33277.45 |
| 4 | 16 | 1 | 33397.52 | 33397.52 |
| 4 | 16 | 2 | 66836.22 | 33418.11 |
| 4 | 16 | 4 | 133589.24 | 33397.31 |

Table 5: `verify_kzg`: sweep over $(\ell, n_{\text{cols}})$ with $L \in \{1, 2, 4\}$, eight rounds per row, naive per-column verification.

At $\ell = 2$, mean time per `verify_column` sits near 16.6 s ($\sim 1.7 \times 10^4$ ms) across both $n_{\text{cols}} \in \{4, 8\}$ and all $L$. At $\ell = 4$ it rises to a band near 33.3 s ($\sim 3.3 \times 10^4$ ms), consistent with doubling the $2\ell$ pairing work per column. For fixed $\ell$, varying $n_{\text{cols}}$ between 8 and 16 leaves per-column latency almost unchanged on this prototype, while mean time per sampling round still grows approximately linearly in $L$ within each geometry.

#### 4.5.8 Pairing-count model (tabular)

| $\ell$ | $L$ | Naive pairings | Batched pairings |
|---|---|---|---|
| 8 | 1 | 16 | 2 |
| 8 | 2 | 32 | 2 |
| 8 | 4 | 64 | 2 |
| 8 | 8 | 128 | 2 |
| 8 | 16 | 256 | 2 |
| 8 | 32 | 512 | 2 |
| 8 | 64 | 1024 | 2 |

Table 6: `pairing_counts`: naive model $2L\ell$ versus an idealised batched verifier (two pairings).

Table 6 highlights the asymptotic benefit of batching at scale: for $\ell = 8$ and $L = 64$, independent verification is modeled as 1024 pairings, whereas a batched check remains at two pairings in this idealised count.

# References


[1] Mathias Hall-Andersen and Mark Simkin and Benedikt Wagner, *Foundations of Data Availability Sampling*. Cryptology ePrint Archive, Paper 2023/1079, 2023, https://eprint.iacr.org/2023/1079.

[2] Ethereum, *PeerDAS*. ethereum.org, 2026, https://ethereum.org/roadmap/fusaka/peerdas/.

[3] Benedikt Wagner and Arantxa Zapico, *A Documentation of Ethereum's PeerDAS*. Cryptology ePrint Archive, Paper 2024/1362, 2024, https://eprint.iacr.org/2024/1362.